

Evaluación Lab 2 - 2026

Instrucciones Generales

- Para codear el parcial solo podrán utilizar algún editor de texto plano o IDE (sin autocompletado con IA). No utilizar teléfonos durante el examen.
 - Pueden buscar información en Google pero no pueden utilizar asistentes de IA de ninguna clase.
 - Pueden utilizar sus propias laptops o alguna máquina del laboratorio que tenga instalado sbt.
 - Pueden consultar y reutilizar el código que escribieron en el laboratorio 2, pero **solo deben entregar los archivos indicados al final**.
 - Solo pueden usar las **librerías** ya incluidas en el build.sbt del esqueleto. No pueden agregar nuevas dependencias.
-

Compilación

Pueden encontrar el código del esqueleto en el siguiente link:

<https://drive.google.com/file/d/1R9z0ftOIWUi6z8waXSHqc9wRylbzDVkk/view?usp=sharing>

El esqueleto contiene dos proyectos: el proyecto principal del parcial y un servidor mock de Reddit (`reddit-mock/`). **Deben tener ambos corriendo al mismo tiempo**, cada uno en una terminal separada.

Paso 1 — Levantar el servidor mock de Reddit

Para evitar que Reddit bloquee las peticiones durante el examen, el esqueleto apunta a un servidor local que simula la API de Reddit.

En una terminal, desde la carpeta `reddit-mock/`:

```
None  
sbt run
```

Deberían ver:

```
None
Fake Reddit API running on http://localhost:8123
Press Ctrl+C (or stop the process) to shut down.
```

El servidor queda corriendo en el puerto 8123 y sirve los posts de los subreddits `r/scala`, `r/programming` y `r/learnpython` desde archivos JSON locales. **No cerrarlo durante el examen.**

Paso 2 — Ejecutar el proyecto

En otra terminal, desde la carpeta raíz del esqueleto:

```
None
sbt run
```

Verán algo como el siguiente output:

```
None
Diccionario cargado: 65 entidades.

Descargando posts de: http://localhost:8123/r/scala/.json

=====
http://localhost:8123/r/scala/.json
=====
Post: "This week in #Scala (Apr 27, 2026)"
Entidades detectadas:
  [ProgrammingLanguage] Scala
Post: "try it out: Scone media player build with Scala.js, Calico, and Electron"
Entidades detectadas:
  [ProgrammingLanguage] Scala
Post: "sbt-steps 0.3.0 released: streamline your CI using the power of sbt!"
Entidades detectadas:
  (sin entidades detectadas)
Post: "Spark observability tool that works for production debugging?"
Entidades detectadas:
  (sin entidades detectadas)
Post: "Rust for Scala and Haskell Developers: A Surprisingly Familiar Journey by
Pawel Szulc"
Entidades detectadas:
  [ProgrammingLanguage] Scala
  [ProgrammingLanguage] Haskell
  [ProgrammingLanguage] Rust
  ...
```

Ejercicio 1 (3 puntos)

La implementación actual de `Analyzer.detectEntities` busca entidades aplicando la misma lógica de matching para todos los tipos:

```
None  
val lowerText = text.toLowerCase  
dictionary.filter(entity => lowerText.contains(entity.text.toLowerCase))
```

Esto genera **falsos positivos**: el texto "I enjoy a good rust coffee and go for a walk" detectaría incorrectamente `Rust` y `Go` como lenguajes de programación; el texto "JavaScript and Java are popular" detectaría `Java` dos veces.

La solución orientada a objetos es **encapsular la lógica de matching dentro de cada entidad** en lugar de aplicar una regla uniforme desde el `Analyzer`.

a) Agregar en `NamedEntity` el método concreto:

```
None  
def matches(text: String): Boolean = ???
```

Implementarlo con **matching case-insensitive** tal que la entidad no sea parte de una secuencia más larga de letras o dígitos. Los siguientes casos deben cumplirse:

Texto	Entidad	¿Match?
"Java is great"	Java	Si
"I use JavaScript"	Java	No
"I love C++ programming"	C++	Si
"(Java) is verbose"	Java	Si

b) Sobreescibir `matches` en `Technology` para hacer matching de **palabra completa, case-sensitive**: `Python` debe detectarse en "Python is great" pero no en "I love python scripting".

c) Sobreescibir `matches` en `Person` con el siguiente comportamiento: hace match si el texto contiene el **nombre completo** (case-sensitive). Por ejemplo, `Person("Martin Odersky")` debe hacer match en `"Martin Odersky visited EPFL"`.

d) Modificar `Analyzer.detectEntities` para que use `entity.matches(text)` en lugar de la lógica actual.

Ejercicio 2 (3 puntos)

En la práctica, no todas las entidades detectadas son igualmente útiles: lugares como `San Francisco` o organizaciones genéricas como `Google` suelen aparecer en demasiados posts y aportan poco al análisis. Al mismo tiempo, el sistema no soporta aún un tipo de entidad relevante para posts de tecnología: las conferencias académicas. En este ejercicio se extiende el sistema para distinguir qué entidades son relevantes, se agrega la nueva rama de jerarquía para eventos, y se implementa un método de detección que combine ambas restricciones.

a) Agregar en `NamedEntity` el siguiente método concreto:

```
None
def isRelevant: Boolean = true
```

Sobreescibir `isRelevant` de la siguiente manera en las clases existentes:

Clase	<code>isRelevant</code>
<code>Organization</code>	<code>false</code>
<code>University</code>	<code>true</code> (sobreescibe a su padre)
<code>Place</code>	<code>false</code>
<code>Technology</code>	<code>false</code>
<code>ProgrammingLanguage</code>	<code>true</code> (sobreescibe a su padre)

`Person` no sobreescibe `isRelevant`.

b) Agregar en `NamedEntity.scala`:

- Clase abstracta `Event(text: String)` que extiende `NamedEntity`. No requiere implementar `entityType` en esta clase.

- Clase concreta `Conference(text: String)` que extiende `Event` con `entityType = "Conference"`. No sobrescribe `isRelevant` ni `matches`.

Crear el archivo `data/conferences.txt` con al menos 5 conferencias (ejemplos: ICFP, Scala Days, Strange Loop, JVM Language Summit, ScalaMatsuri). Agregar la carga de este archivo en `Dictionary.loadAll()`.

c) Agregar en `Analyzer`:

```
None
def detectRelevant(text: String, dictionary: List[NamedEntity]): List[NamedEntity]
= ???
```

Este método debe retornar únicamente las entidades cuyo `isRelevant` es `true` y cuyo `matches` retorna `true` para el texto dado. `detectRelevant` no reemplaza a `detectEntities` — ambos métodos deben coexistir. `detectEntities` sigue siendo el que detecta todas las entidades sin filtrar.

Ejercicio 3 (2 puntos)

El formato actual de salida lista todas las entidades detectadas en un bloque plano, lo que dificulta la lectura cuando hay muchos tipos distintos. En este ejercicio se reemplaza ese formato por uno agrupado por tipo y se integra en el flujo principal del sistema.

a) Implementar en `Formatters`:

```
None
def formatGroupedNERResult(postTitle: String, entities: List[NamedEntity]): String
= ???
```

El método debe formatear las entidades detectadas **agrupadas por tipo**, con el siguiente formato:

```
None
Post: "Odersky at ICFP: Scala's future is bright"
Entidades detectadas:
  Conference (1):
```

```
ICFP
Person (1):
  Odersky
ProgrammingLanguage (1):
  Scala
```

Requisitos del formato:

- Los grupos deben aparecer **ordenados alfabéticamente** por `entityType`.
- El encabezado de cada grupo debe incluir la **cantidad de entidades** de ese tipo:
`Conference (1):.`
- Dentro de cada grupo, las entidades deben estar **ordenadas alfabéticamente** por `text`.
- Si no se detectaron entidades, retornar el mismo mensaje que `formatNERResult:`
`(sin entidades detectadas)`.

b) Modificar `Main.scala` para que use `detectRelevant` junto con `formatGroupedNERResult` al mostrar el resultado de cada post, en lugar de `formatNERResult`. Las estadísticas globales al final deben seguir calculándose sobre todas las entidades detectadas por `detectEntities` (sin filtrar por relevancia).

Entrega

Para entregar el examen deben subir los siguientes archivos a través del formulario provisto. Deberán generar un archivo comprimido que incluya a todos:

- `NamedEntity.scala`
- `Analyzer.scala`
- `Formatters.scala`
- `Main.scala`
- `data/conferences.txt`

Para generar el archivo comprimido pueden usar un comando similar al siguiente:

```
None
tar -czvf parcial2.tar.xz data/conferences.txt src/main/scala/NamedEntity.scala
src/main/scala/Analyzer.scala ...
```

Deberán recibir un mail de confirmación, no se retiren del aula hasta recibirlo ya que no se aceptarán reclamos.

Form: <https://forms.gle/YzvAhnQXE8VnHW6x6>